

MPC模型预测控制

实际代码mpc.c中没有使用优化器、没有障碍物
Obstacle

邱高 | 2025年7月10日修改

实现目的：帮助车辆周期性基于当前传感器收集的测量信息在线求解一个有限时间路径调整避障问题。

MPC介绍

MPC (Model Predictive Control, 模型预测控制) 是一种将“模型 + 预测 + 优化控制”深度结合的先进控制方法：

- 模型 (Model)：** MPC 使用系统的动态数学模型（如车辆运动模型、路径拟合模型）来预测**未来 N 个时间步内**系统的状态演化。
- 预测 (Predictive)：** 在每个控制步骤，MPC 向前预测未来轨迹，考虑当前状态和外部扰动，并评估多种可能的控制策略。
- 控制 (Control)：** 通过求解带有约束（如安全距离、最大转角、加速度限制）的优化问题，MPC 计算出一系列控制动作（如转向角、加速度）使系统跟随目标轨迹，同时避障最优。执行时仅应用第一个控制量，并在下一时刻重新开始优化，实现“滚动时域”控制。

这种方法的优势在于：

- 预判未来、提前应对：**能够预测即将发生的碰撞风险并规划避障动作；
- 多约束适应性强：**可同时处理路径跟踪、动态响应、安全距离、物理限幅等多种限制；
- 跨多个领域：**广泛应用于化工过程、无人车辆、电力系统和机器人等

MPC 预测模型状态与控制变量

状态变量与控制变量

状态变量

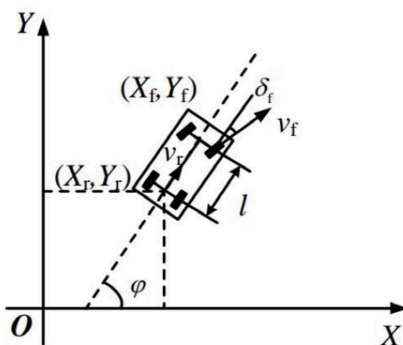
车辆的每一个预测时刻 t 包含以下状态变量

状态变量	含义	举例
x	当前车辆在地图中的 x 坐标位置	如 0
y	当前车辆在地图中的 y 坐标位置	如 0
psi	当前车辆的航向角（朝向）	如 0（正前方）
v	当前速度（单位：m/s）	如 2.0
cte	当前车辆与参考路径的横向偏差	$cte = f(x) - y$
epsi	当前车辆朝向与路径切线的角度差	$epsi = psi - atan(f'(x))$

控制变量

- δ (delta)：前轮转角（弯道转向）
- a：加速度（或减速）

状态转移方程（预测模型）



车辆运动模型

使用的是 车辆的运动学模型 (Kinematic Model) , 忽略了动力学中的摩擦力等复杂因素。核心思想是: 根据当前状态, 加上控制输入 (方向盘角度、加速度) , 预测下一时刻的状态。

代码块

```
1 fg[1 + x_start + t] = x1 - (x0 + v0 * cos(psi0) * dt);
```

- 意义: 预测下一个 x 坐标 x1 应该等于当前坐标 x0 加上沿着车辆朝向 psi0 的移动距离。
- $v0 * \cos(\psi_0) * dt$: 车辆在当前方向上的前进距离。

代码块

```
1 fg[1 + y_start + t] = y1 - (y0 + v0 * sin(psi0) * dt);
```

- 意义: 预测下一个 y 坐标。
- $v0 * \sin(\psi_0) * dt$: 车辆在垂直方向上的前进分量。

代码块

```
1 fg[1 + psi_start + t] = psi1 - (psi0 + v0 * delta0 / Lf * dt);
```

- 意义: 预测航向角 psi1。
- 控制变量 delta0 是方向盘转角。
- Lf 是车辆的前轴中心到质心的距离, 转向半径的关键参数。
- $v0 * \delta_0 / L_f$ 近似表示角速度。

代码块

```
1 fg[1 + v_start + t] = v1 - (v0 + a0 * dt);
```

- 意义: 预测速度 v1。
- a0 是加速度输入。

代码块

```
1 fg[1 + cte_start + t] = cte1 - ((f0 - y0) + v0 * sin(eps0) * dt);
```

- cte1 是下一步的横向偏差 (cross track error) 。
- f0 是在当前 x0 下的参考路径高度 (f(x0)) 。
- $f_0 - y_0$ 是当前横向偏差。
- $v0 * \sin(\epsilon_0) * dt$ 是由于方向偏差引起的偏移。

代码块

```
1 fg[1 + epsi_start + t] = epsi1 - ((psi0 - psides0) + v0 * delta0 / Lf * dt);
```

- epsi1 是下一个步骤的航向误差。
- $\psi_0 - \psi_{sides0}$ 是当前误差。
- $v0 * \delta_0 / L_f * dt$ 是由于控制输入引起的方向变化。

时间维度设计

代码块

```
1 size_t N = 400; // 预测步数 (400 步)
2 double dt = 0.05; // 每步间隔 (50ms)
```

MPC 会预测未来 20 秒 (400×0.05) 内车辆的状态，时间跨度较长，适合展示避障能力。

Cost 函数设计逻辑

参考状态误差项

代码块

```
1 fg[0] += 5 * CppAD::pow(vars[cte_start + i] - ref_cte, 2);
2 fg[0] += 5 * CppAD::pow(vars[epsi_start + i] - ref_epsi, 2);
3 fg[0] += CppAD::pow(vars[v_start + i] - ref_v, 2);
```

- **横向偏差 CTE**: 表示车辆当前位置相对于参考路径在横向上的距离偏差，权重为 5，鼓励车辆尽可能贴近参考轨迹；
- **航向误差 EPSI**: 表示车辆行驶方向与路径切线方向的差异，权重也为 5，鼓励车辆方向对齐参考路径方向；
- **速度误差 V**: 表示当前速度与设定参考速度 ref_v 的偏差，权重为 1，鼓励车辆匀速行驶，不快也不慢。

控制输入代价项

代码块

```
1 fg[0] += CppAD::pow(vars[delta_start + t], 2);
2 fg[0] += CppAD::pow(vars[a_start + t], 2);
```

- **方向盘转角惩罚**: 对转角 delta 的绝对值进行惩罚（权重 1），鼓励车辆尽量保持方向稳定、少打方向盘，提升行驶平稳性；
- **加速度惩罚**: 对加速度 a 的绝对值进行惩罚（权重 5），鼓励车辆避免剧烈加速或减速，实现更舒适的驾驶；
- **高速打方向惩罚**: 对转角与速度的乘积 $\delta * v$ 进行惩罚（权重 5），主要目的是限制在高速状态下剧烈转向，降低侧翻风险，保障车辆稳定性与安全性。

控制输入变化代价项

代码块

```
1 fg[0] += 200 * CppAD::pow(vars[delta_start + i + 1] - vars[delta_start + i], 2);
2 fg[0] += 10 * CppAD::pow(vars[a_start + i + 1] - vars[a_start + i], 2);
```

- **方向盘转角变化惩罚**: 这一项对连续两个时间步之间的方向盘转角差值进行惩罚（权重为 200），即约束 delta 的变化速率，防止方向盘快速来回摆动，提升车辆操控的平滑性和乘坐舒适性；
- **加速度变化惩罚**: 这一项对连续两个时间步之间的加速度差值进行惩罚（权重为 10），鼓励加速和减速的过程平稳有序，避免频繁突然变速，从而进一步提升驾驶舒适性并降低能源消耗或机械负担。

参考路径与障碍物设计

参考路径设计

代码块

```
1 const int num_points = 50;
2 VectorXd ptsx(num_points);
3 VectorXd ptsy(num_points);
4 for (int i = 0; i < num_points; ++i) {
5     double x = i * 10.0;
6     double y = x * 0.5; // 一条直线 y = 0.5x
7     ptsx[i] = x;
8     ptsy[i] = y;
9 }
10 auto coeffs = polyfit(ptsx, ptsy, 1); // 拟合为一阶多项式 y = 0.5x
```

定义了 50 个参考路径点，x 每隔 10 增加一次， $y = 0.5x$ 构成一条直线轨迹。通过 polyfit 拟合这些点得到一阶多项式 coeffs，作为 MPC 控制器的目标轨迹，用于后续横向误差 (cte) 和航向误差 (epsi) 的计算。

障碍物设计

随机生成障碍物：

代码块

```
1  std::vector<Obstacle> generate_obstacles(const Eigen::VectorXd &state)
2  {
3      std::vector<Obstacle> new_obstacles;
4      double cx = state[0];
5      double cy = state[1];
6
7      int num_obstacles = 12;
8      double radius_min = 5.0;
9      double radius_max = 20.0;
10
11     for (int i = 0; i < num_obstacles; ++i)
12     {
13         double angle = ((double)rand() / RAND_MAX) * 2 * M_PI;
14         double radius = radius_min + ((double)rand() / RAND_MAX) * (radius_max - radius_min);
15         double obs_x = cx + radius * cos(angle);
16         double obs_y = cy + radius * sin(angle);
17
18         // 可选：设置障碍物速度为0（静态）或随机（动态）
19         bool is_dynamic = false; // 或随机决定
20         double vx = 0.0;
21         double vy = 0.0;
22         if (is_dynamic)
23         {
24             double v = ((double)rand() / RAND_MAX) * 1.0; // 0~1 m/s
25             double theta = ((double)rand() / RAND_MAX) * 2 * M_PI;
26             vx = v * cos(theta);
27             vy = v * sin(theta);
28         }
29
30         new_obstacles.push_back({obs_x, obs_y, vx, vy, is_dynamic});
31     }
```

模拟现实中存在的静态或动态障碍物，通过 MPC 的代价函数与约束项实现避障。用于**基于当前车辆状态动态生成障碍物**，返回一个 `std::vector<Obstacle>` 类型的障碍物列表。函数接收车辆当前状态 `state`（为一个 Eigen 向量），从中提取当前位置 `(cx, cy)`，然后围绕该点在一个半径范围内（5 到 100 米）**随机分布生成 250 个障碍物**。障碍物的位置通过随机角度和半径计算得到 `(obs_x, obs_y)` 坐标。障碍物默认设为**静态**（`is_dynamic = false`），其速度 `(vx, vy)` 设置为 0，但也可以根据需要启用动态障碍物功能。最终，将生成的障碍物封装为 `Obstacle` 结构体加入 `new_obstacles` 向量并返回。

障碍物预测

代码块

```
1  AD<double> obs_x = obs.x + (obs.dynamic ? obs.vx * t * dt : 0);
2  AD<double> obs_y = obs.y + (obs.dynamic ? obs.vy * t * dt : 0);
```

每个时间步对所有障碍物进行预测。如果 `obs.dynamic == true`，说明是动态障碍物，使用**简单线性预测**（假设动态障碍物以恒定速度沿直线运动，未来的位置用当前位置加上速度乘以时间来线性推算）；

否则是静态障碍物。

经过的障碍物忽略

代码块

```
1  AD<double> angle_diff = heading - psi;
2      while (angle_diff > M_PI)
3          angle_diff -= 2 * M_PI;
4      while (angle_diff < -M_PI)
5          angle_diff += 2 * M_PI;
6      if (CppAD::abs(angle_diff) > M_PI_2)
7      {
8          ++constraint_idx;
```

```
9         continue; // 超过 ±90°, 视为后方, 跳过
10     }
```

判断障碍物是否位于车辆前方 ±90° 视野范围内，决定是否将该障碍物纳入代价函数或约束中参与避障计算。具体来说，heading 表示从车辆当前位置指向障碍物的方向角，而 psi 是车辆当前的航向角，二者之差 angle_diff 表示障碍物相对于车辆前进方向的角度偏差。由于角度是周期性的，代码通过循环将 angle_diff 归一化到 $[-n, n]$ 范围内，避免误判。接着判断其绝对值是否超过 $\frac{n}{2}$ （即 ±90°），若是则说明障碍物在车辆侧后方，不具备直接碰撞风险，因此通过 continue 跳过该障碍物的处理。这样可以有效地**减少优化问题的复杂度**，聚焦于前方真正有可能发生碰撞的障碍物，提高避障效率与减少算力浪费。

计算车辆与障碍物的欧氏距离

代码块

```
1  AD<double> dist = CppAD::sqrt(CppAD::pow(px - obs_x, 2) + CppAD::pow(py - obs_y,
```

采用欧式距离进行计算：

$$\text{dist} = \sqrt{(x_{\text{car}} - x_{\text{obs}})^2 + (y_{\text{car}} - y_{\text{obs}})^2}$$

障碍物位置 obs_x, obs_y 是未来某个时刻预测的；px, py 是 MPC 对车辆当前位置的预测。每一帧都对所有障碍物计算一次当前距离

计算车辆与障碍物的相对速度

代码块

```
1  AD<double> heading = CppAD::atan2(dy, dx); // dy = obs_y - py
2  AD<double> psi = vars[psi_start + t];      // 车辆的航向角
3  AD<double> rel_vx = vars[v_start + t] * CppAD::cos(psi) - obs.vx;
4  AD<double> rel_vy = vars[v_start + t] * CppAD::sin(psi) - obs.vy;
5  AD<double> rel_speed = CppAD::sqrt(CppAD::pow(rel_vx, 2) + CppAD::pow(rel_vy, 2))
```

车辆在全球坐标系下的速度被分解为两个分量：

$\text{vars}[v_{\text{start}} + t] * \cos(\text{psi})$ ：表示车辆在 x 方向的速度，

$\text{vars}[v_{\text{start}} + t] * \sin(\text{psi})$ ：表示在 y 方向的速度。

将这两个分量分别减去障碍物在对应方向的速度 vx 和 vy，即可得到车辆与障碍物之间的相对速度向量。最后，通过计算该向量的模，即：

$$v_{\text{rel}} = \sqrt{(v_{\text{car},x} - v_{\text{obs},x})^2 + (v_{\text{car},y} - v_{\text{obs},y})^2}$$

得到了车辆相对于障碍物的速度大小，用于避障代价函数的计算。该值越大，表示车辆以更高的速度逼近障碍物，系统会给予更高的代价以促使其规避。

避障策略（硬约束 + 软代价）

硬约束：保持最小安全距离

代码块

```
1  constraints_lowerbound[idx] = safe_dist; // 最小安全距离
2  constraints_upperbound[idx] = 1.0e19;
```

保证了 MPC 预测轨迹中每个点与障碍物之间的欧式距离必须大于 safe_dist，否则优化器会认为该轨迹不可行（infeasible），从而强制规避碰撞。这是“硬约束”的体现，起到了刚性防撞作用。

软代价项：融合相对速度的惩罚

代码块

```
1  // Sigmoid权重, 距离越远权重越低, safe_dist附近权重接近1
2  AD<double> avoid_weight = 1.0 / (1.0 + CppAD::exp(sigmoid_k * (dist - safe_dist))
3  // 代价叠加
4  fg[0] += avoid_scale * avoid_weight * clipped_rel_speed;
```

计算了一个基于**sigmoid函数**的“避让权重”，其中：

- `dist` 是车辆到障碍物的欧式距离；
- `safe_dist` 是安全距离阈值；
- `sigmoid_k` 是控制曲线陡峭程度的参数（越大越陡峭）；
- 输出范围在 $(0, 1)$ 之间，距离小于安全距离时，权重逼近 1；距离远于安全距离时，权重迅速下降。

基于 `sigmoid` 和相对速度的避障代价项，更稳定、物理意义更强且对动态障碍物更敏感。相比之下

代码块

```
1 fg[0] += 5.0 / (CppAD::pow(dist, 2) + 1.0);
```

容易在距离接近障碍物时引发优化问题的震荡或失败，且无法反映“静态障碍物 vs 远离车辆的动态障碍物”之间的实际风险差异。

自适应机制和Fallback热加载机制

自适应机制

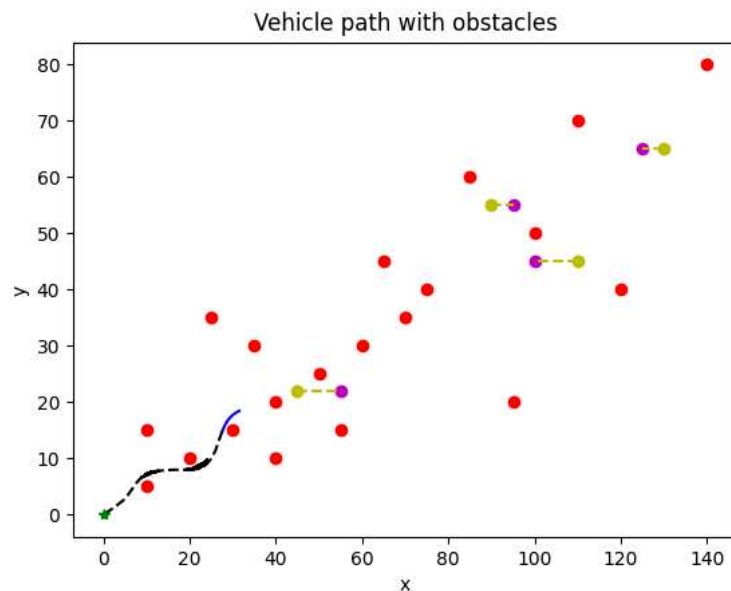
在复杂障碍物或不良初始条件下，MPC 可能求解失败（如 IPOPT 状态码非 1）。长预测步长 `N` 会增加优化变量维度、加重求解器负担，导致难以收敛。

代码块

```
1 while (mpc_result.status != 1 && N > N_min)
2 {
3     N = std::max(N / 2, N_min); // 每次减半, 直到 N_min
4     ...
5     mpc_result = mpc.Solve(...); // 重新求解
6 }
```

Fallback热加载机制

即使减小 `N`，MPC 仍有可能失败。为了避免系统“冻结”或“重置”，**fallback 热加载：利用最近一次成功求解的历史控制量，推进当前状态，形成“软重启”**。如下图中黑色线条加重区域。



采用fallback效果图

外层循环：尝试回溯过去状态

代码块

```
1 const int max_fallback_attempts = 5;
2 int fallback_attempts = 0;
3
4 while (mpc_result.status != 1 && !success_states.empty() && fallback_attempts <
```

每次尝试都从历史的 success_states、success_vars 中回退一组控制量和对应状态，一旦发现该组控制量非法或推进后状态异常，就会跳过并回退更早的一次，若 5 次尝试均失败，视为 fallback 全部失败。

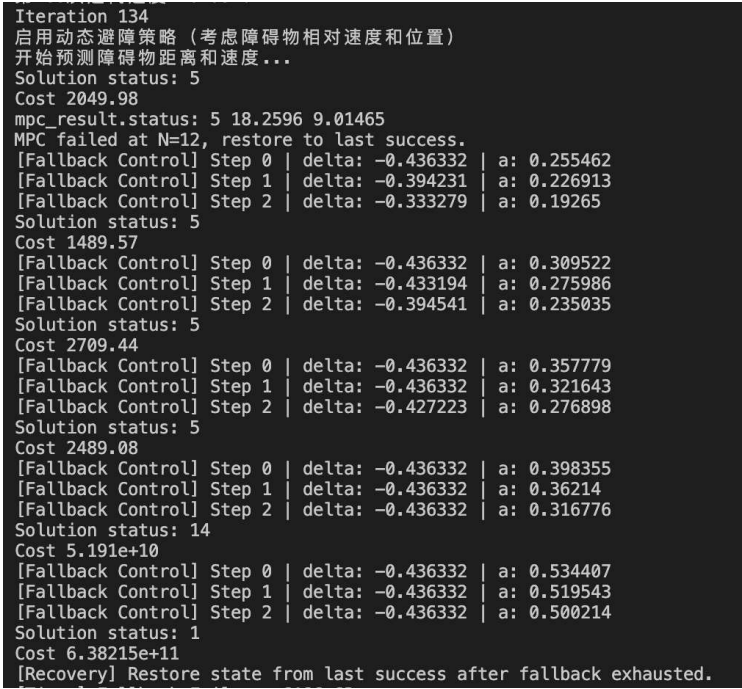
内层推进：推进回溯的状态

代码块

```
1  int max_push_steps = std::min(3, static_cast<int>(restore_N - 1));
2  ...
3  for (int i = 0; i < max_push_steps; ++i)
```

模拟如果用上一次成功解（的前若干步控制量）去控制车辆，能不能把车辆“带出当前卡死状态”。但由于历史解并不总是适用于当前场景，需要模拟一小段实际使用，验证控制是否正常。

下图是预定50步，实际200步，24个障碍物情况。其中一次FallBack5次尝试均失败的耗时。

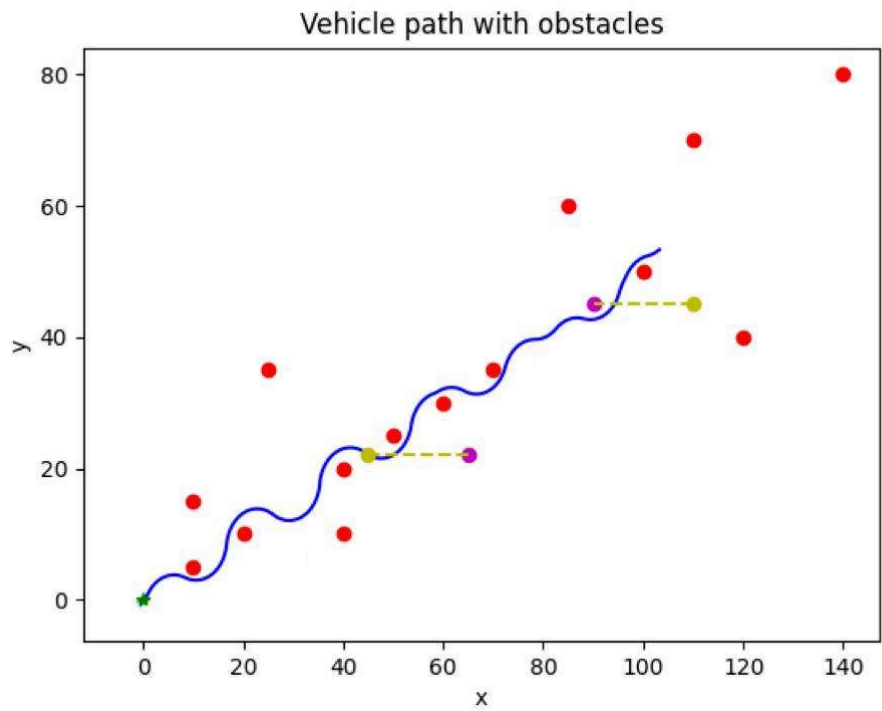


Fallback连续5次失败消耗时间

仿真可视化结果

车辆避障仿真模拟

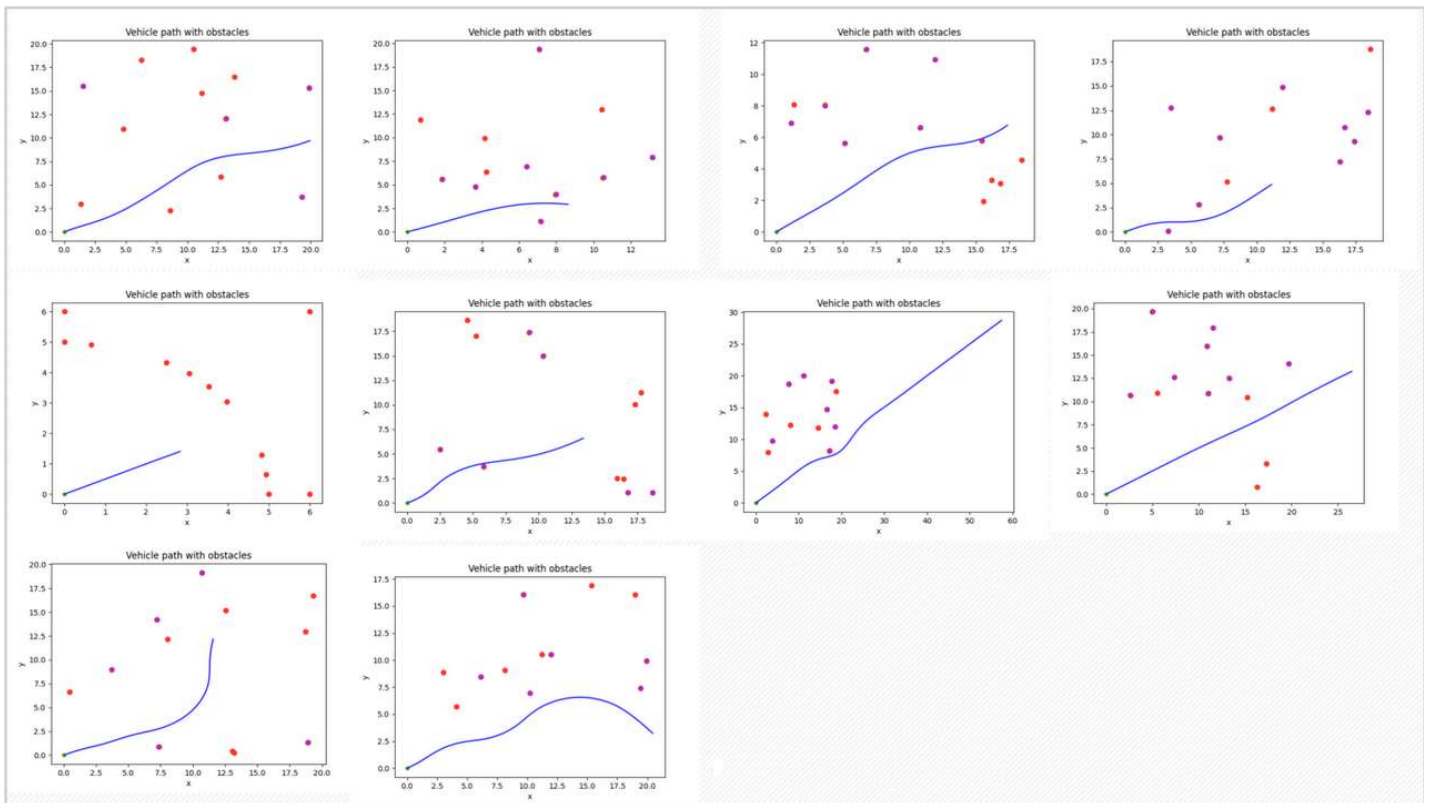
元素	含义说明
蓝色曲线	车辆预测路径, MPC 求解结果
红色圆点	15 个静态障碍物, 从文本文件加载
起点绿色星号	起始状态 (x=0, y=0.5)
黄色圆点	动态障碍物的起始位置
紫色圆点	动态障碍物的终止位置
黄色虚线	动态障碍物的运行轨迹
黑色虚线	车辆真实行驶路线
曲线路径	明显体现了车辆在进行多次绕障操作
上下摆动	每次绕障操作均体现为路径偏离中心线并回正



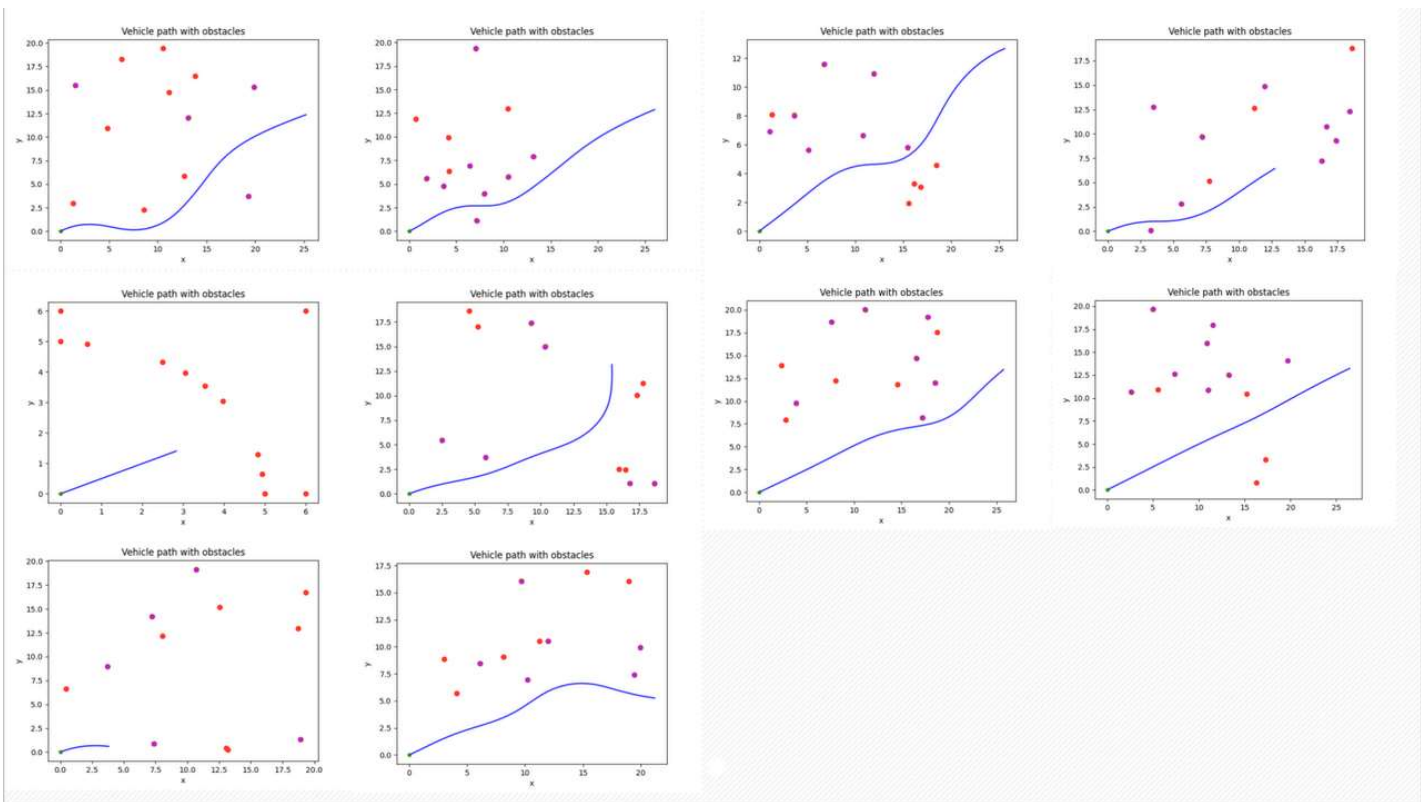
初始点预测后面400步的路线规划

图展示了基于 MPC（模型预测控制）方法下的车辆避障仿真结果。蓝色曲线为车辆的预测行驶路径，绿色星号表示车辆起点，红色圆点为从文本中加载的 15 个静态障碍物和两个动态障碍物。车辆初始速度较低 ($v=5.0$)，并通过延长预测步数 ($N=400$)，实现了对未来 20 秒内轨迹的规划。在避障过程中，MPC 同时引入了硬约束（保证车辆与障碍物的最小安全距离）和软代价对靠近障碍物的行为施加惩罚），使车辆能够灵活绕开路径上的多个障碍物。

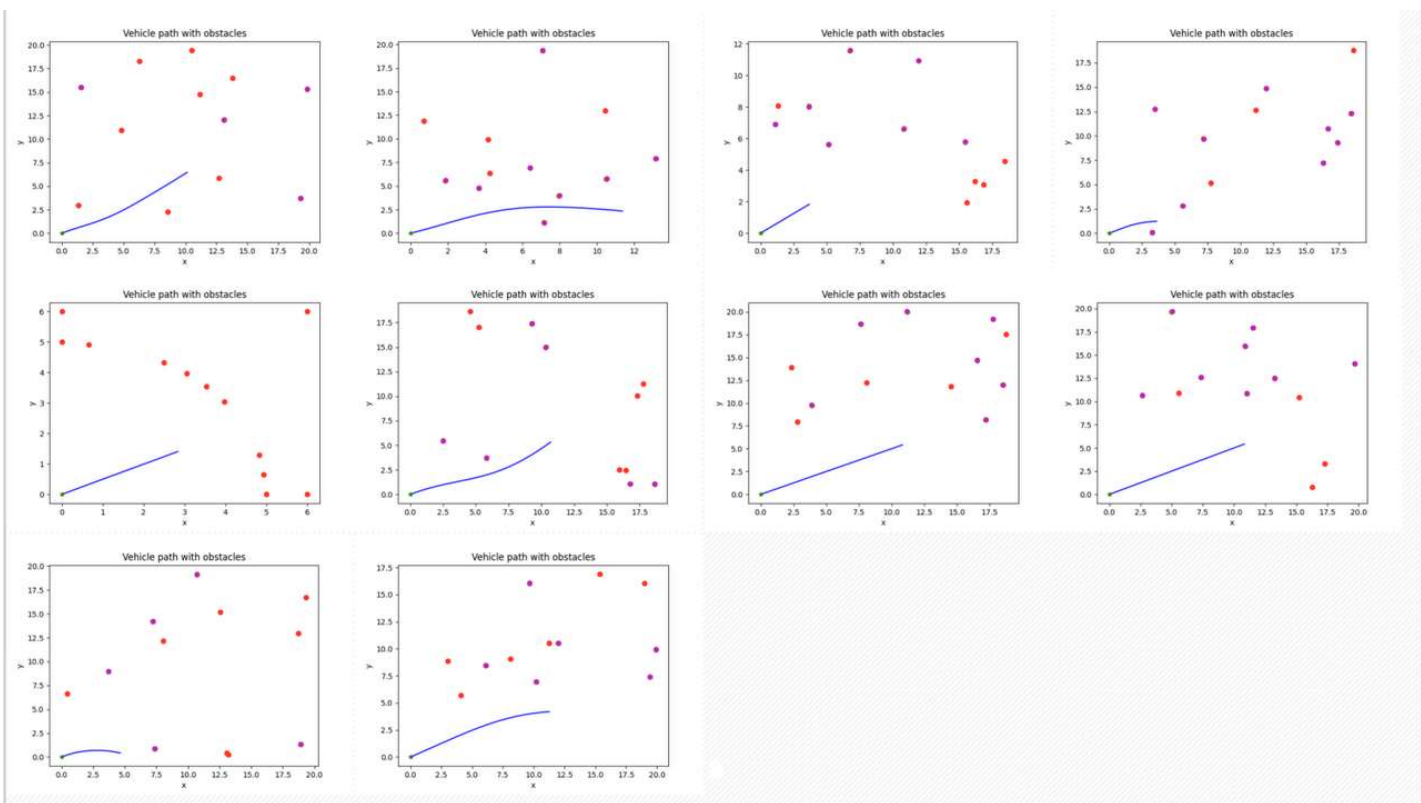
预测步长对识别速度的影响



预测步长400效果图



预测步长50效果图



预测步长25效果图

	测试1时耗	测试2时耗	测试3时耗	测试4时耗	测试5时耗	测试6时耗	测试7时耗	测试8时耗	测试9时耗	测试10时耗
步长400	5718.62 ms	6551.47 ms	5346.25 ms	33338.8 ms	3280.48 ms	5095.39 ms	8298.81 ms	5866.37 ms	3716.39 ms	4769.65
步长50	746.234 ms	606.006 ms	726.304 ms	352.126 ms	123.064 ms	205.313 ms	271.246 ms	253.195 ms	141.168 ms	182.685
步长25	97.6773 ms	147.645 ms	65.8083 ms	99.9522 ms	89.9928 ms	105.765 ms	63.3504 ms	76.4098 ms	127.661 ms	66.6235

算法的性能要求每秒处理几百到几千字节，障碍物的基本信息如下：

代码块

```

1  struct Obstacle
2  {
3      double x;
4      double y;
5      double vx;
6      double vy;
7      bool dynamic;
8  };

```

● MPC模型预测控制

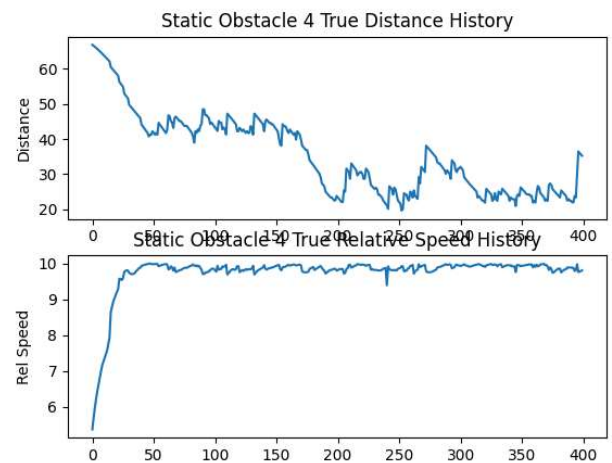
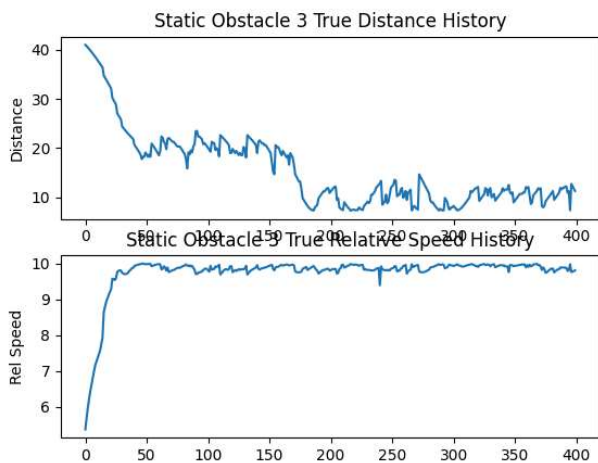
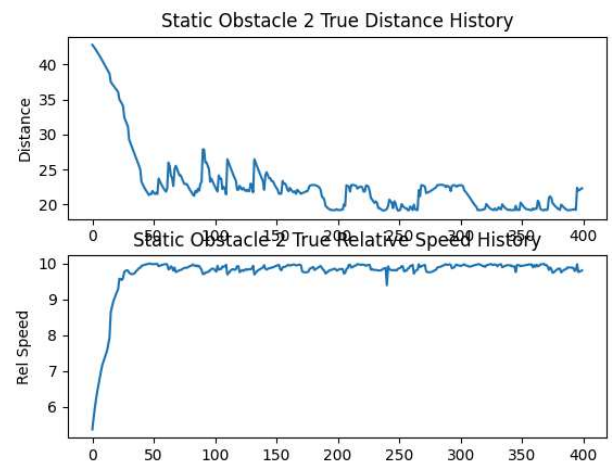
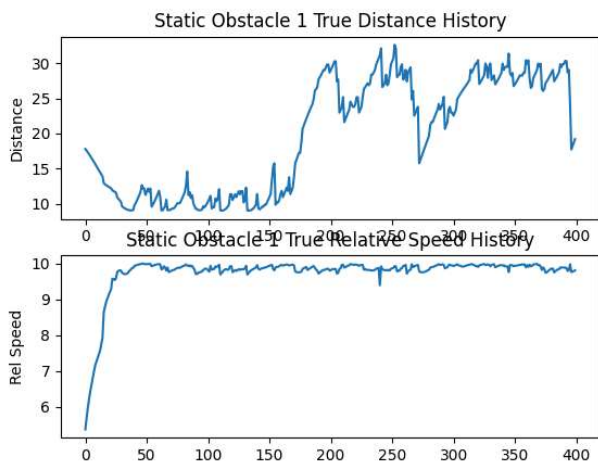
例如前障碍物的距离和相对速度等信息 每秒几百到几千字节

每秒几十到几百字节

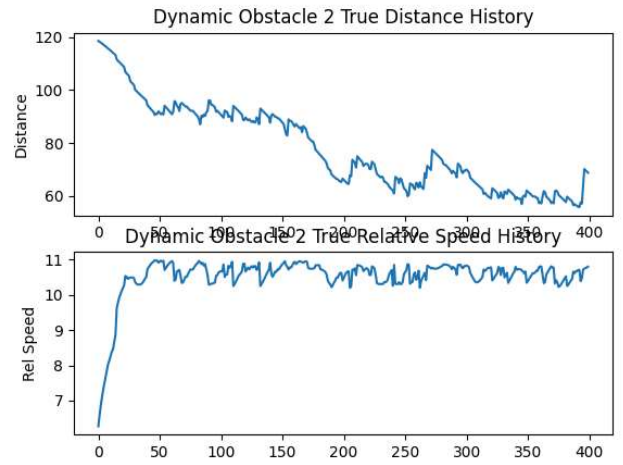
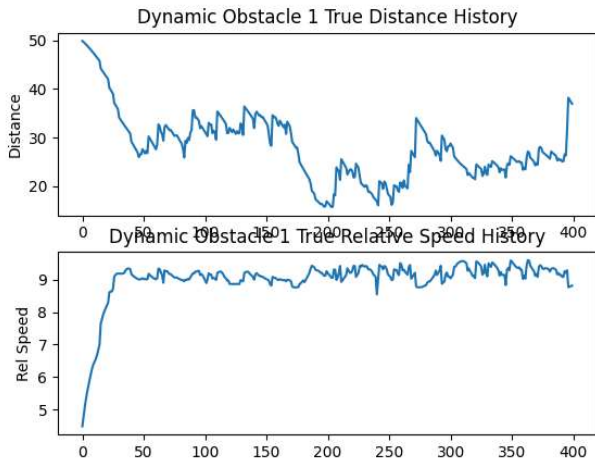
Double字长8个字节，bool进行补偿对齐之后，每个障碍物的信息40字节。极限情况每秒需要处理9999个字节的数据（每秒处理250个障碍物），那么在一个步长50ms内，车辆需要完成对于新出现大概12个障碍物的预测。上述测试步长下无法满足这一条件，需要进一步优化。

障碍物测距和相对速度测量

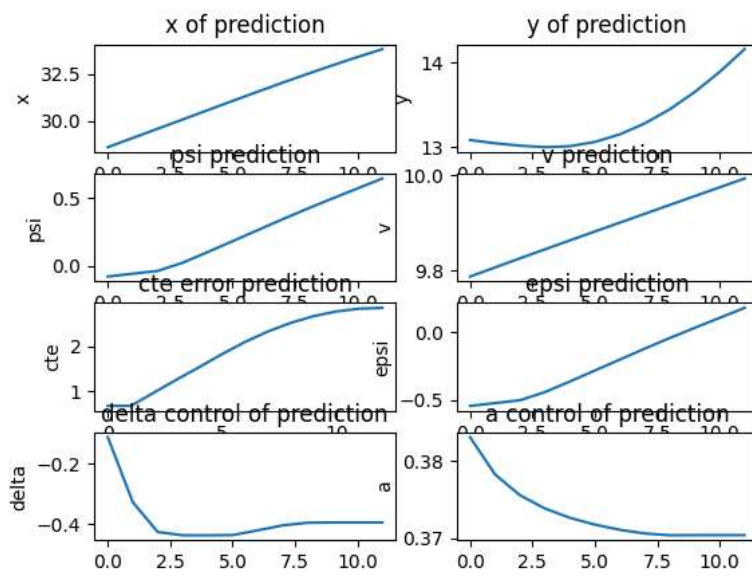
静态障碍物



动态障碍物



车辆状态



车辆在整个仿真过程中每一步的历史状态和控制量，即车辆实际运动的历史轨迹和控制输入。

[项目地址](#)